

שאלה 1.

סעיף א' - פונקציית האלגוריתם:

```
def approximate_shapley(players, cost_function, iterations=10000):
    """
    :param players: 3, 2, 1] (למשל)
    :param cost_function: פונקציה שמקבלת קבוצת שחקנים ומחזירה את העלות שלהם
    :param iterations: ככל שגבוה יותר, הדיוק עולה - (M) מספר הדגימות
    :return: מילון עם ערך שאפלי המקורב לכל שחקן
    """
    # אתחול סכום העלויות השוליות לכל שחקן
    total_marginal_costs = {player: 0.0 for player in players}

    for _ in range(iterations):
        # 1. בחירת פרמוטציה אקראית P
        p = list(players)
        random.shuffle(p)

        current_set = set()

        # 2. מעבר על השחקנים לפי הסדר בפרמוטציה וחישוב עלות שולית
        for player in p:
            # עלות לפני הצטרפות השחקן
            cost_before = cost_function(current_set)

            # הוספת השחקן לקבוצה
            current_set.add(player)

            # עלות אחרי הצטרפות השחקן
            cost_after = cost_function(current_set)

            # העלות השולית היא ההפרש
            marginal_contribution = cost_after - cost_before

            # צבירת העלות השולית לשחקן
            total_marginal_costs[player] += marginal_contribution

        # 3. חישוב הממוצע לכל שחקן
        shapley_values = {}
        for player, total in total_marginal_costs.items():
            shapley_values[player] = total / iterations
```

```
return shapley_values
```

הפונקציה מקבלת כקלט רשימת שחקנים, פונקציית חישוב העלויות, ומספר איטרציות לביצוע.
(הוספתי קובץ [main.py](#) בהגשה עם נקודת כניסה לסקריפט ודוגמה של המטוסים)

הסבר:

הפונקציה משתמשת בrandom בשביל לעשות פרמוטציה על הרשימת שחקנים, במקום לבדוק את כל האפשרויות (שזה ח!) אנחנו דוגמים תת-קבוצה אקראית של סדרים.
בכל איטרציה, אנחנו בונים את הקבוצה הנוכחית, לאותה פרמוטציה current_set. בכל שלב השחקן הבא בתור "רואה" את אלו שהיו לפניו כקבוצה קיימת.

העלות השולית, marginal_contribution מחושבת לפי ההגדרה -

$$c(S \cup \{j\}) - c(S)$$

בנוסף עבור פרמטר האיטרציות שמת 10,000 כברירת מחדל.

סעיף ב' -

עבור הדוגמה עם המטוסים ומסלולי ההמראה, שמתי שלושה ערכים:

מטוס א' - 100

מטוס ב' - 200

מטוס ג' - 300

ביקשתי מבינה מלאכותית של גוגל (Gemini) לחשב לי את כל הפרמוטציות עבור אותם ערכים (במקום לעשות את זה ידנית מאפס):

להלן השיחה:

<https://gemini.google.com/share/ebd4f07d584c>

מהצ'אט נובע שהערכים עבור כל מטוס הם (בהתאמה):

א' - 33.33

ב' - 83.33

ג' - 183.33

התחלתי לבצע הרצות של האלגוריתם הנל עם כמויות שונות של איטרציות.
להלן התוצאות.

עבור 10 איטרציות:

```
• > python ./main.py  
Approximate Shapley Values:  
Plane_A: 40.00  
Plane_B: 80.00  
Plane_C: 180.00
```

עבור 20 איטרציות:

```
• > python ./main.py  
Approximate Shapley Values:  
Plane_A: 15.00  
Plane_B: 60.00  
Plane_C: 225.00
```

עבור 100 איטרציות:

```
• > python ./main.py  
Approximate Shapley Values:  
Plane_A: 31.00  
Plane_B: 84.00  
Plane_C: 185.00
```

מטוס 2 נמצא כאן בטווח הפרש של פחות מ-1%, בעוד ששני המטוסים האחרים עדיין סוטים מההפרש הנדרש.

עבור 1000 איטרציות:

```
• > python ./main.py  
Approximate Shapley Values:  
Plane_A: 32.60  
Plane_B: 84.90  
Plane_C: 182.50
```

עכשיו מטוס א' בטווח, מטוס ב' בטווח ומטוס ג' בטווח.
ננסה להוריד.

עבור 500 איטרציות:

```
● > python ./main.py
Approximate Shapley Values:
Plane_A: 36.20
Plane_B: 88.20
Plane_C: 175.60
```

:600

```
● > python ./main.py
Approximate Shapley Values:
Plane_A: 32.00
Plane_B: 84.33
Plane_C: 183.67
```

:700

```
● > python ./main.py
Approximate Shapley Values:
Plane_A: 32.43
Plane_B: 84.57
Plane_C: 183.00
```

:750

```
● > python ./main.py
Approximate Shapley Values:
Plane_A: 32.93
Plane_B: 90.67
Plane_C: 176.40
```

:800

```
● > python ./main.py
Approximate Shapley Values:
Plane_A: 33.50
Plane_B: 80.75
Plane_C: 185.75
```

נעלה דרסטית, ל10 אלף איטרציות:

```

● > python ./main.py
Approximate Shapley Values:
Plane_A: 33.07
Plane_B: 84.28
Plane_C: 182.65

```

סכום הפרשים: $1.89 = 0.26 + 0.95 + 0.68$

20 אלף:

```

● > python ./main.py
Approximate Shapley Values:
Plane_A: 32.89
Plane_B: 83.44
Plane_C: 183.68

```

סכום הפרשים: $0.9 = 0.44 + 0.11 + 0.35$

30 אלף:

```

● > python ./main.py
Approximate Shapley Values:
Plane_A: 33.25
Plane_B: 83.04
Plane_C: 183.71

```

סכום הפרשים:

$0.75 = 0.08 + 0.29 + 0.38$

ב3 ההרצות האחרונות קל לראות שכל הערכים הם בטווח של לכל היותר אחוז אחד הפרש מהתוצאות בפועל (בפרט, ההרצה האחרונה מניבה את התוצאות הכי הדוקות) מסקנה היא שככל שנעשה יותר איטרציות נגיע לסכום יותר קרוב למקורי. אם נרצה לעצור בטווח של אחוז אחד - 20 אלף איטרציות זה מספיק אם נרצה להעלות כמה שיותר - אז ככל שיותר איטרציות יותר טוב.

ביקשתי מ־chatGPT ליצור לי רשימה של פייתון המורכבת מ־100 מספרים עם ערכים בין 10 ל־300 (בשביל לסמלץ מרחקי מסלולי המראה שונים).
לאחר מכן ביקשתי ממנו לשכפל חלק מהערכים באקראי, למספר אקראי של שחקנים. למשל: באקראיות שלושה שחקנים עם אותו ערך, או חמישה עם אותו ערך וכו'.
תוך כדי אמרתי לו שישים את הערכים ברשימה של מפתח-ערך (dictionary)
להלן הקישור:

<https://chatgpt.com/share/695a72ac-0494-800f-b085-9bb069465858>

כעת, אני ביקשתי מ־Gemini, באותה שיחה של מקודם, לחשב את הערכים של הרשימה שה־chatGPT הביא.
<https://gemini.google.com/share/2c6146ba9b02>

הדבקתי את המילון:

```
players_n = {  
    "Player_1": 214,  
    "Player_2": 57,  
    "Player_3": 289,  
    "Player_4": 143,  
    "Player_5": 98,  
    "Player_6": 271,  
    "Player_7": 34,  
    "Player_8": 186,  
    "Player_9": 220,  
    "Player_10": 115,  
    "Player_11": 67,  
    "Player_12": 299,  
    "Player_13": 154,  
    "Player_14": 201,  
    "Player_15": 83,  
    "Player_16": 176,  
    "Player_17": 248,  
}
```

להלן הפונקציה של סעיף ג:

```
items = list(players_n.items())  
  
for n in range(10,101, 10):  
    current_batch = dict(items[:n])  
    players_list = list(current_batch.keys())  
  
    def runway_cost_function(subset):  
        if not subset:  
            return 0  
        return max(current_batch[player] for player in subset)  
  
    result = approximate_shapley(players_list, runway_cost_function, iterations=10000)  
    for player, value in result.items():  
        print(f"{player}: {value:.2f}")  
    print(f"Batch End: {n}")  
    print(f"/-----/")
```

(זה עשה לי קצת בעיות אז הגדרתי את פונקציית חישוב המסלול המראה בתוך הלולאה הפנימית, על מנת שהפונקציה תמיד תיגש ל־individual cost עבור הח הנוכחי)
חישוב של כמות האיטרציות על מנת חישוב דיוק מירבי הוא מאוד גבוהה.
מכיוון שככל ש־n גדל, מרחב הפרמוטציות גדל אקספוננציאלית.

אבל מכיוון שאנחנו מדברים על קירוב, אם נלך לפי ההיגיון - עבור 3 שחקנים אחוז הקירוב היה בטווח של ~אחוז אחד הפרש מהמציאות על ידי 20,000 איטרציות.
ככל הנראה שכמות כזו של איטרציות (אולי קצת יותר - למשל 50,000) תיתן אחוז דיוק די טוב.

לצורך השוואה, הנה כמה batches של האלגוריתם אל מול החישובים בפועל של הבינה מלאכותית:

שחקנים 1-10:

```
≡ shapley_for_n_players.txt
```

```
1 Player_1: 36.43
2 Player_2: 5.92
3 Player_3: 78.01
4 Player_4: 17.79
5 Player_5: 10.80
6 Player_6: 61.18
7 Player_7: 3.40
8 Player_8: 26.22
9 Player_9: 35.32
10 Player_10: 13.94
11 Batch End: 10
12 /-----/
```

Batch 1 (Players 1-10)

- Player 1: 33.78
- Player 2: 5.96
- Player 3: 79.28
- Player 4: 18.18
- Player 5: 11.08
- Player 6: 61.28
- Player 7: 3.4
- Player 8: 26.78
- Player 9: 35.78
- Player 10: 13.51

מצד ימין תשובה של הבינה המלאכותית, מצד שמאל החישוב בפועל עבור 10 אלף איטרציות

שחקנים 1-30:

Batch 3 (Players 1-30)

- Player 1: 12.08 | Player 2: 1.96 | Player 3: 29.34 | Player 4: 6.12 | Player 5: 3.7
- Player 6: 22.51 | Player 7: 1.13 | Player 8: 9.19 | Player 9: 12.94 | Player 10: 4.55
- Player 11: 2.36 | Player 12: 39.34 | Player 13: 6.81 | Player 14: 10.58 | Player 15: 3.03
- Player 16: 8.36 | Player 17: 17.61 | Player 18: 3.37 | Player 19: 5.47 | Player 20: 21.01
- Player 21: 1.59 | Player 22: 10.28 | Player 23: 8.13 | Player 24: 2.04 | Player 25: 26.84
- Player 26: 4.86 | Player 27: 3.6 | Player 28: 11.58 | Player 29: 7.34 | Player 30: 1.27

```
55 Player_21: 1.56
56 Player_22: 10.63
57 Player_23: 7.63
58 Player_24: 2.04
59 Player_25: 27.39
60 Player_26: 4.22
61 Player_27: 3.84
62 Player_28: 11.22
63 Player_29: 7.26
64 Player_30: 1.21
65 Batch End: 30
```

```
35 Player_1: 12.12
36 Player_2: 1.99
37 Player_3: 29.47
38 Player_4: 5.59
39 Player_5: 3.66
40 Player_6: 21.97
41 Player_7: 1.12
42 Player_8: 8.84
43 Player_9: 13.34
44 Player_10: 4.80
45 Player_11: 2.40
46 Player_12: 39.41
47 Player_13: 6.79
48 Player_14: 10.28
49 Player_15: 2.95
50 Player_16: 8.60
51 Player_17: 18.64
52 Player_18: 3.47
53 Player_19: 5.92
54 Player_20: 20.63
```

תמונה עליונה תשובה של הבינה, שתי תמונות תחתונות חישוב בפועל עבור 10 אלף איטרציות (עבור חלק מהשחקנים הדיוקים ממש קטנים - שחקנים 2, 7, 24, 21 בעוד שעבור חלק הפספוסים יותר גדולים - שחקנים 8, 9, 17, 23..)

ננסה להעלות 20 אלף איטרציות

אותן דוגמאות:

שחקנים 1-10:

Batch 1 (Players 1-10)

- Player 1: 33.78
- Player 2: 5.96
- Player 3: 79.28
- Player 4: 18.18
- Player 5: 11.08
- Player 6: 61.28
- Player 7: 3.4
- Player 8: 26.78
- Player 9: 35.78
- Player 10: 13.51

```
1 Player_1: 34.45
2 Player_2: 5.77
3 Player_3: 77.99
4 Player_4: 18.17
5 Player_5: 10.75
6 Player_6: 61.40
7 Player_7: 3.47
8 Player_8: 27.26
9 Player_9: 36.07
10 Player_10: 13.67
11 Batch End: 10
```

קל לראות שעבור חלק מהשחקנים הדיוק ירד - שחקנים 2, 3, לעומת דוגמה קודמת, בעוד שעבור שחקנים כמו שחקן 1, ו-4, הדיוק עלה.

שחקנים 1-30:

Batch 3 (Players 1-30)

- Player 1: 12.08 | Player 2: 1.96 | Player 3: 29.34 | Player 4: 6.12 | Player 5: 3.7
- Player 6: 22.51 | Player 7: 1.13 | Player 8: 9.19 | Player 9: 12.94 | Player 10: 4.55
- Player 11: 2.36 | Player 12: 39.34 | Player 13: 6.81 | Player 14: 10.58 | Player 15: 3.03
- Player 16: 8.36 | Player 17: 17.61 | Player 18: 3.37 | Player 19: 5.47 | Player 20: 21.01
- Player 21: 1.59 | Player 22: 10.28 | Player 23: 8.13 | Player 24: 2.04 | Player 25: 26.84
- Player 26: 4.86 | Player 27: 3.6 | Player 28: 11.58 | Player 29: 7.34 | Player 30: 1.27

Player_16: 8.27	Player_1: 12.26
Player_17: 17.84	Player_2: 1.94
Player_18: 3.36	Player_3: 28.36
Player_19: 5.22	Player_4: 6.26
Player_20: 21.45	Player_5: 3.88
Player_21: 1.58	Player_6: 22.71
Player_22: 10.45	Player_7: 1.22
Player_23: 7.88	Player_8: 9.50
Player_24: 2.04	Player_9: 13.22
Player_25: 26.68	Player_10: 4.70
Player_26: 4.75	Player_11: 2.34
Player_27: 3.55	Player_12: 39.60
Player_28: 10.93	Player_13: 6.55
Player_29: 7.79	Player_14: 10.26
Player_30: 1.25	Player_15: 3.17

הדיוק עבור השחקנים שצינו קודם - 2, 7, 24, 21 (לפי הסדר): ירד במעט, ירד, מדויק, ירד באחוז.
עובר שחקנים 8, 9, 17, 23 : ירד, גדל במעט, גדל בהרבה, גדל במעט.